

FANUC TP Programming Quick Reference

EE0849 Introduction to Robotics

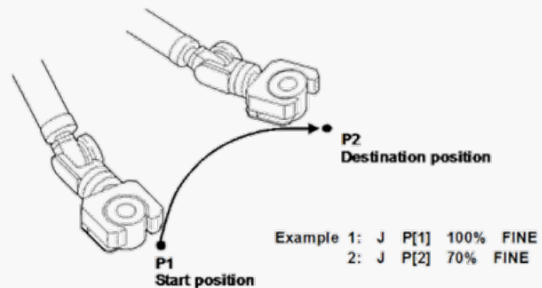
Basri Erdogan

Motion Types

Type	Syntax	Speed Units
Joint (J)	J P[i] 100% FINE	% (1–100)
Linear (L)	L P[i] 500mm/sec FINE	mm/sec, cm/min
Circular (C)	C P[i] P[j] 200mm/sec FINE	mm/sec, cm/min
Arc (A)	A P[i] 200mm/sec FINE	mm/sec, cm/min

Motion Format

- **Joint motion (J)**
 - All joints move together and stops at the same time. The path of motion is usually non-linear.



EE0848 - Machine Learning for SP

4

Figure 1: Joint vs Linear motion

- **J** – all joints move together, curved path. Use for large free moves.
- **L** – straight-line TCP path. Use near workpieces.

- **C** – arc through a via point + target (both taught in one instruction).
- **A** – chained arcs from 3+ consecutive arc instructions.

Termination: FINE vs CNT

```
L P[1] 100mm/sec FINE    ; stop at point
L P[2] 500mm/sec CNT50   ; blend through
```

- FINE – stops exactly at target. **Use at pick/place.**
- CNT 0 – closest approach without stopping.
- CNT 100 – maximum smoothing, fastest. **Use for transit.**

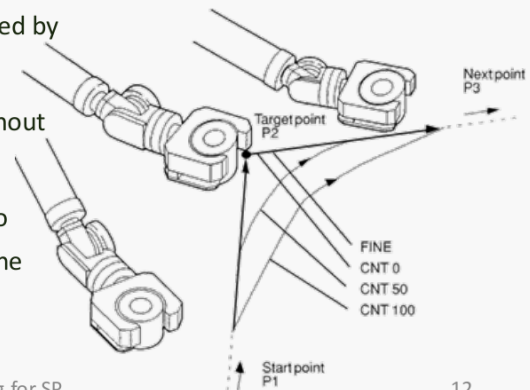
Positioning Path

- FINE positioning path

- Robot stops at target point before moving to the next target point.

- CNT positioning path

- When CNT specified robot approaches but does not stop at the target and moves to the next point.
 - How closely robot approach to target can be defined by a value from 0 to 100.
 - When 0, it goes as close to target as possible without stopping.
 - When 100 robot moves along the farthest path to target, because robot does not decelerate near the target.



EE0848 - Machine Learning for SP

12

Figure 2: FINE vs CNT path behavior

Speed Control

Motion	Unit	Range	Example
Joint	%	1–100	J P[1] 50% FINE
Linear	mm/sec	1–2000	L P[2] 800mm/sec CNT50
Linear	cm/min	1–12000	L P[2] 4800cm/min CNT50
Timed	sec	0.1–3200	L P[3] 2.0sec FINE

Override: OVERRIDE=50% ; limits all motion to 50% of programmed speed. Can use register: OVERRIDE=R[10]% ;

Position Data

Cartesian (UF/UT dependent):

UF:2 UT:1

X: 1500.374 W: 10.000

Y: -342.992 P: 20.000

Z: 956.895 R: 40.000

CONF: N, U, T, 0, 0, 0

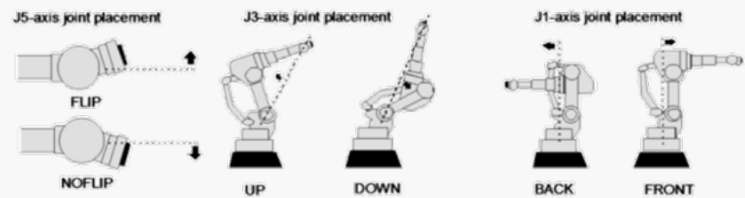
Joint (absolute angles):

J1: 45.000 J4: 0.000

J2: -30.000 J5: -90.000

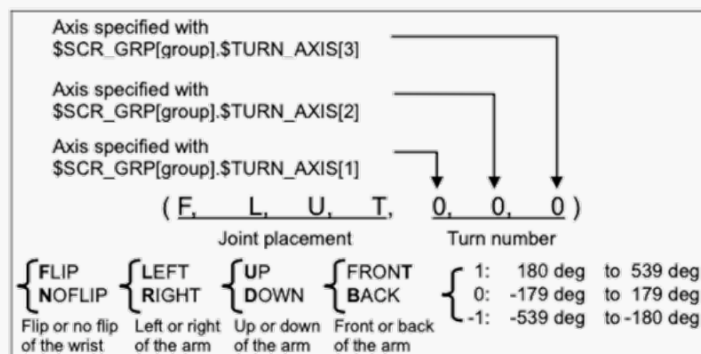
J3: 60.000 J6: 180.000

Position Data



- **Configuration**

- Configuration represents the orientation of the robot.
- Several configuration available which meet the condition of Cartesian coordinates (x,y,z,w,p,r)



EE0848 - Machine Learning for SP

9

Figure 3: Configuration flags

Configuration: N/F = NOFLIP/FLIP (wrist), U/D = UP/DOWN (elbow), T/B = FRONT/BACK (reach), then turn numbers for J4/J5/J6.

Registers

Numeric Registers R[i] (R[1]–R[200]):

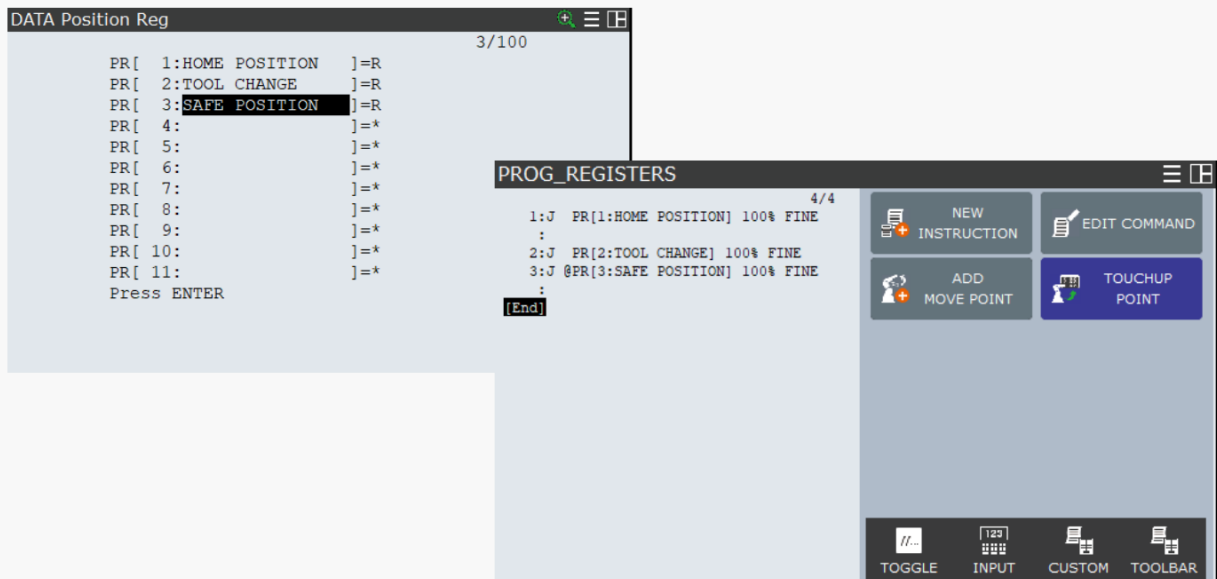
```
R[1]=0 ; ; set to zero
R[1]=R[1]+1 ; ; increment
R[3]=R[1]*2 ; ; arithmetic
```

```
R[5]=R[1] MOD 3 ; ; modulo
```

Position Registers PR[i] (PR[1]–PR[100]):

```
PR[1]=P[1] ; ; copy position
PR[1]=LPOS ; ; current position
PR[1,1]=100.0 ; ; set X component
PR[1,3]=PR[1,3]+50 ; ; shift Z
L PR[1] 500mm/sec FINE ;
```

Position Register (Example)



EE0848 - Machine Learning for SP

20

Figure 4: Position Registers on pendant

P[i] vs PR[i]: P[i] is local to one program (taught waypoints). PR[i] is global across all programs (computed positions, offsets).

PR element index: 1=X, 2=Y, 3=Z, 4=W, 5=P, 6=R (Cartesian) or 1=J1 ... 6=J6 (Joint).

Conditional Logic

IF with Jump:

```
IF DI[1]=ON, JMP LBL[10] ;
IF R[1]>5, JMP LBL[20] ;
```

IF/THEN/ELSE (V8.30+):

```
IF (DI[1]=ON) THEN ;
  CALL GRIP_CLOSE ;
```

```
ELSE ;  
    CALL GRIP_OPEN ;  
ENDIF ;
```

SELECT (Case):

```
SELECT R[1]=1, JMP LBL[1] ;  
      =2, JMP LBL[2] ;  
      =3, JMP LBL[3] ;  
      ELSE, JMP LBL[99] ;
```

Operators: =, <>, >, <, >=, <=

Combinators: AND, OR (cannot mix)

Loops

FOR loop:

```
FOR R[1]=1 TO 10 ;  
  L P[1] 500mm/sec CNT100 ;  
ENDFOR ;
```

WHILE pattern (LBL/IF/JMP):

```
LBL[1] ;  
  IF R[1]>=10, JMP LBL[2] ;  
  ; --- body ---  
  R[1]=R[1]+1 ;  
  JMP LBL[1] ;  
LBL[2] ;
```

DO-WHILE pattern:

```
LBL[1] ;  
  ; --- body (runs at least once) ---  
  R[1]=R[1]+1 ;  
  IF R[1]<10, JMP LBL[1] ;
```

WAIT instructions:

```
WAIT 1.00(sec) ;  
WAIT DI[1]=ON ;  
WAIT DI[1]=ON TIMEOUT,LBL[99] ;
```

I/O Instructions

```
DO[1:Gripper]=ON ; ; set digital output  
DO[1:Gripper]=OFF ; ; clear digital output  
DO[2:Done]=PULSE,0.5sec ; ; single timed pulse  
WAIT DI[1:Part_Sensor]=ON ; ; wait for digital input  
IF DI[1]=ON, JMP LBL[10] ; ; branch on input
```

I/O Types: DI/DO (digital), RI/RO (robot), GI/GO (group), AI/AO (analog), UI/UO (user/safety).

Frames

User Frame (UFRAME) – workpiece location:

```
UFRAME_NUM=2 ; ; switch frame
```

Set via 3-point method on pendant. If part moves to new fixture, re-teach the frame – not every point.

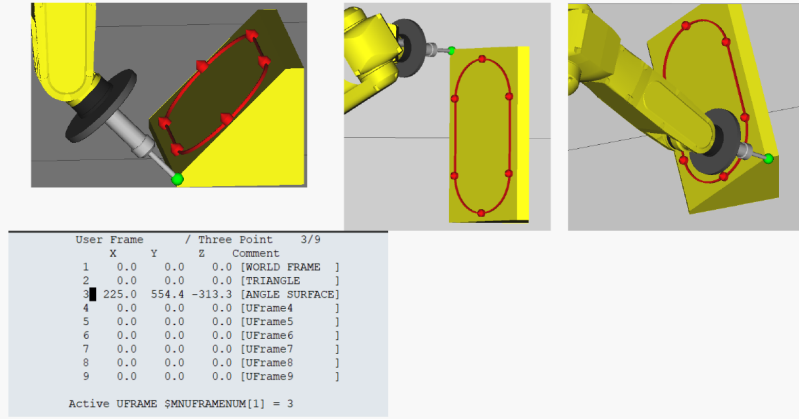
Tool Frame (UTOOL) – TCP offset:

```
UTOOL_NUM=3 ; ; switch tool
```

Set via direct entry (measured offset) or 3-point method. Wrong UTOOL = robot goes to wrong place.

User frame setup

- Let's set the user frame using 3 point method



EE0848 - Machine Learning for SP

16

Figure 5: User Frame setup via 3-point method in Roboguide

Offsets

Manual PR math:

```
PR[10]=P[1:Base] ;  
PR[10,1]=PR[10,1]+50.0 ; ; X+50  
L PR[10] 200mm/sec FINE ;
```

General Offset (User Frame relative):

```
L P[1] 500mm/sec FINE Offset,PR[10] ;
```

Tool Offset (Tool Frame relative):

```
L P[1] 500mm/sec FINE  
Tool_Offset,PR[20] ;
```

Approach/retract along tool Z regardless of orientation.

OFFSET CONDITION (set once, apply many):

```
OFFSET CONDITION PR[10] ;  
L P[1] 500mm/sec FINE Offset ;  
L P[2] 500mm/sec FINE Offset ;
```

Subroutines

```
CALL PICK ; ; run subprogram  
CALL MOVE_TO(1) ; ; pass AR[1]=1
```

Break programs into reusable parts. Each subprogram is a separate /PROG.

Program Creation (Roboguide)

Create a Program



Figure 6: Creating a program

Linear and Joint motion (Example)

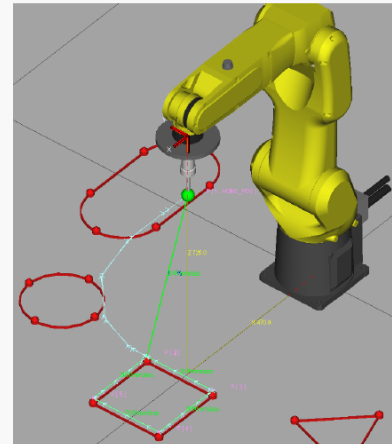
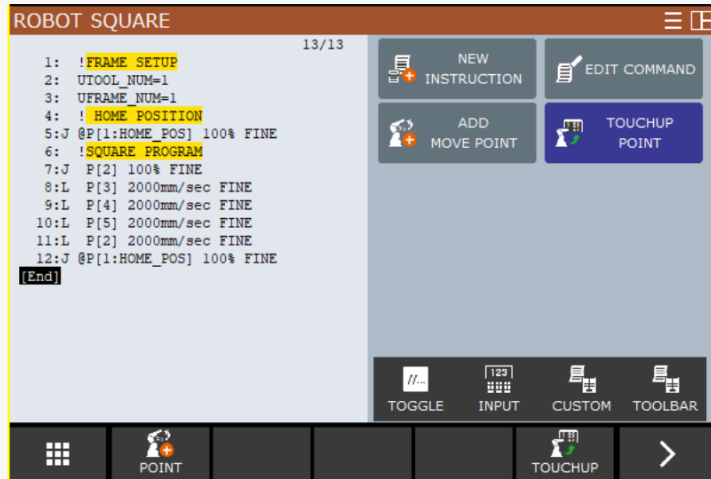


Figure 7: TP editor with Roboguide 3D view

Example: Complete Pick & Place

```
/PROG PICK_PLACE
/MN
1: UFRAME_NUM=1 ;
2: UTOOL_NUM=1 ;
3: DO[1:Gripper]=OFF ;
4: J P[1:Home] 100% FINE ;
5: LBL[1] ;
6: WAIT DI[1:Part_Sensor]=ON ; ; wait for part
7: L P[2:Pick_App] 500mm/sec FINE ; ; approach
8: L P[3:Pick] 100mm/sec FINE ; ; contact
9: DO[1:Gripper]=ON ; ; grip
10: WAIT .30(sec) ;
11: IF DI[2:Grip_OK]=OFF, JMP LBL[99] ; ; verify
12: L P[2:Pick_App] 500mm/sec FINE ; ; retract
13: L P[4:Place_App] 500mm/sec CNT50 ;
14: L P[5:Place] 100mm/sec FINE ;
15: DO[1:Gripper]=OFF ;
16: WAIT .20(sec) ;
17: L P[4:Place_App] 500mm/sec FINE ;
18: JMP LBL[1] ; ; loop
19: LBL[99] ;
20: DO[3:Error_Light]=ON ;
```

```
21: PAUSE ;  
/END
```

Pendant Quick Navigation

Action	Path
Create program	SELECT
Edit instruction	[INST] button
I/O monitor	MENU > I/O > Digital
Position registers	DATA > Position Reg
Frames setup	MENU > 6 SETUP > 3 Frames
Single step	STEP button (toggle)
Speed override	+% / -% buttons